

Relaciones entre objetos

Introducción

Los objetos se comunican, se relacionan entre sí, enviándose mensajes.

Cuando los objetos se envían mensajes entre sí para ejecutar una operación, el objeto receptor responde con otro mensaje: el resultado de esa operación que se ejecutó.

En este intercambio de mensajes se generan relaciones y las relaciones entre objetos se clasifican de acuerdo a diferentes criterios, basados en las siguientes 3 formas fundamentales:

1. Cardinalidad.
2. Direccionalidad.
3. Ordinalidad.

Cardinalidad: concepto

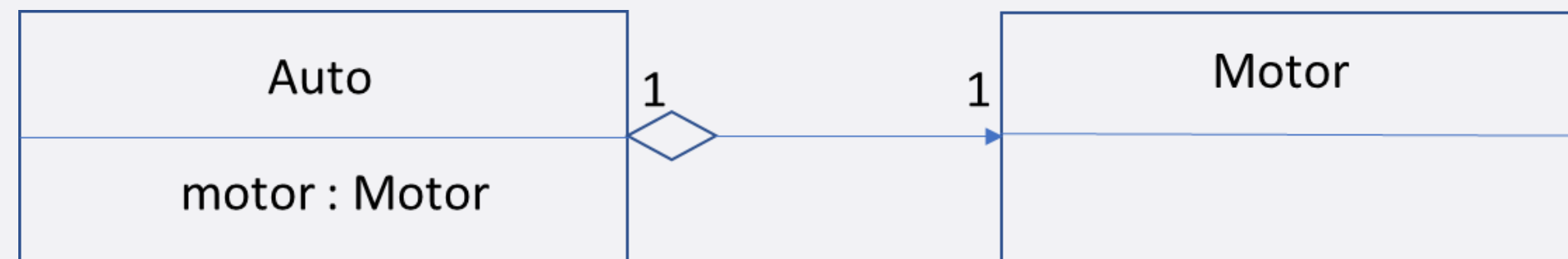
Da cuenta de cuántos objetos existen de cada lado de la relación.

Considerando una relación binaria entre el juego de entidades A y el B, la cardinalidad puede ser:

- Uno a uno.
- Uno a muchos o muchos a uno.
- Muchos a muchos.

Cardinalidad: uno a uno

Una entidad en A está asociada con cualquier número (0 o más) de entidades en B:



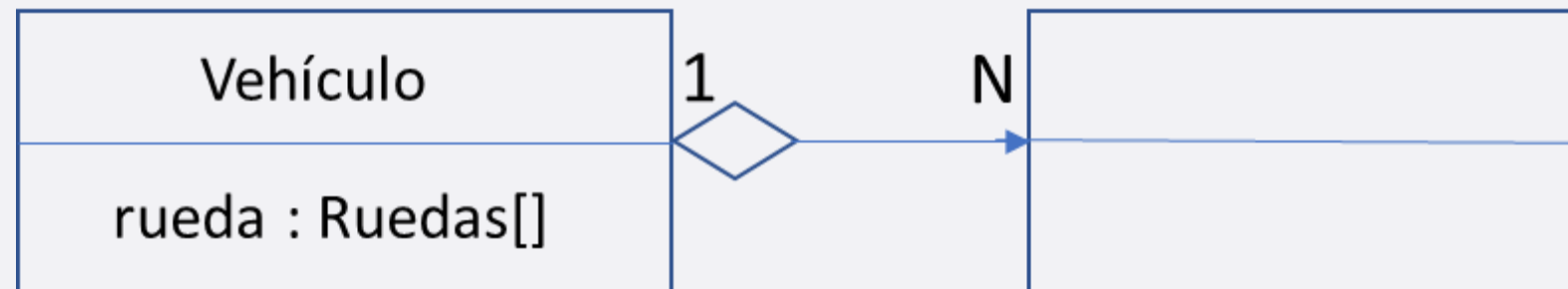
Ejemplo de código: Un Auto tiene un solo Motor.

```
Auto x
1 package edu.up.online.relaciones.unoAuno;
2
3 /**
4  * Relacion 1-a-1 de Auto con Motor
5  */
6 public class Auto {
7
8     Motor motor;
9
10    int velocidadMaxima;
11    String marca;
12 }
```

```
Motor x
1 package edu.up.online.relaciones.unoAuno;
2
3 public class Motor {
4
5     private String fabricante;
6     private String potencia;
7
8 }
```

Cardinalidad: uno a muchos

Una entidad en A está asociada con cualquier número (0 o más) de entidades en B:

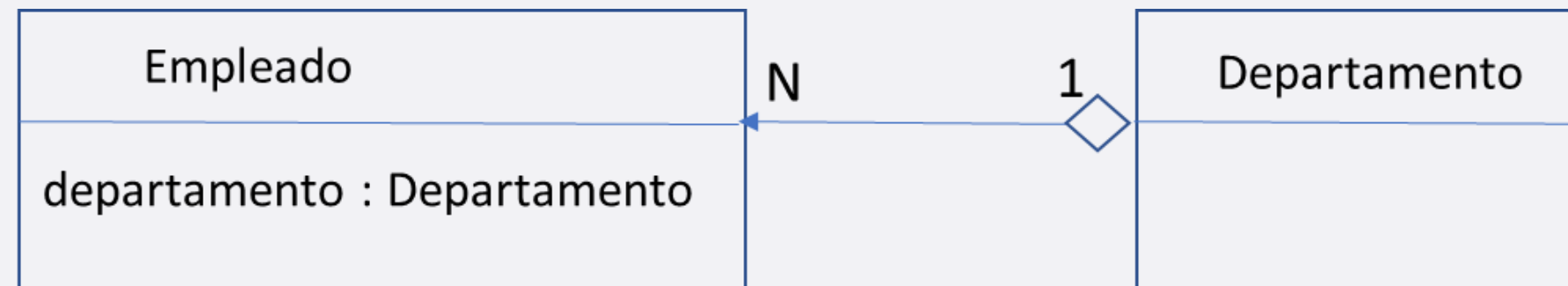


Ejemplo de código: un vehículo puede tener uno o más ruedas.

```
Vehicle x
1 package edu.up.online.relaciones.unoAmuchos;
2
3 /**
4  * Relacion 1-a-N
5  * Ruedas puede ser un arreglo de longitud:
6  * - 0 (un vehículo sin ruedas, ej. un aerodeslizador)
7  * - 1 un monociclo
8  * - 2 una bicilceta, una moto
9  * - 3 un triciclo
10 * - 4 un auto
11 * - 6 un camión
12 * - etc.....
13 */
14 public class Vehiculo {
15
16     Rueda[] ruedas;
17
18     String fabricante;
19     float precio;
20 }
```

Cardinalidad: muchos a uno

Una entidad en A está asociada con cualquier número (0 o más) de entidades en B:

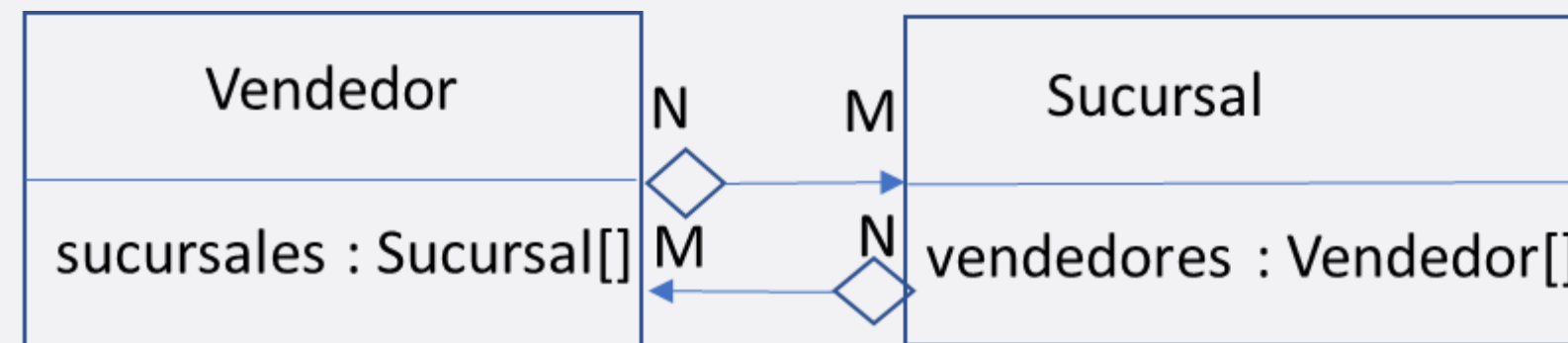


Ejemplo de código: se supone que un empleado trabaja en un departamento (si se es responsable del área de sueldos, no se puede ser responsable del área de seguridad informática).

```
1 package edu.up.online.relaciones.muchosAUno;
2
3 public class Empleado {
4
5     String nombre;
6     Departamento departamentoActual;
7 }
```

Cardinalidad: muchos a muchos

Una entidad en A está asociada con cualquier número de entidades en B (0 o más) y viceversa.



Ejemplo de código I: Los vendedores deben rotar por una serie de sucursales, una vez por mes.

```
1 package edu.up.online.relaciones.muchosAmuchos;
2
3 public class Vendedor {
4
5     String nombre;
6     Sucursal[] sucursalesAsignadas;
7
8 }
```

Ejemplo de código II: Una sucursal tiene un equipo de vendedores. Cada uno de ellos puede tener una serie de sucursales asignadas por las que debe rotar.

```
1 package edu.up.online.relaciones.muchosAmuchos;
2
3 public class Sucursal {
4
5     Vendedor[] equipoVendedores;
6
7 }
8
```

Direccionalidad: concepto

Relación unidireccional: para cada objeto de un lado de la relación le corresponden uno o más objetos del otro lado de la relación.

Relación bidireccional: para cada objeto de un lado de la relación le corresponden muchos objetos del otro lado y viceversa.

Direccionalidad: ejemplos

Relación unidireccional

Ejemplo 1: en una empresa, un empleado puede tener asignado una serie de vehículos que puede manejar. Se quiere saber qué vehículos tiene asignados en un determinado momento.

Ejemplo 2: en una empresa, un empleado puede tener asignado una serie de vehículos que una persona puede manejar; pero el negocio puede no necesitar esa información. En cambio, únicamente, que a partir de un empleado se pueda saber qué vehículo tiene asignado en un determinado momento.

Relación bidireccional

Ejemplo: disponemos de un listado que contiene la nómina completa de una empresa y, por el otro lado, tenemos un listado de vehículos de esa misma empresa. En función de lo anterior, precisamos conocer, por un lado, qué vehículo maneja una persona “X” de esa empresa y, a su vez, quién tiene asignado un determinado vehículo.

Ordinalidad: concepto

Es un indicador del orden, el peso y/o la importancia de la relación entre objetos.

De acuerdo a esa importancia, podemos estar frente a una relación de **agregación** o de **composición**.

- **Agregación:** es el caso en el que un objeto A “usa” a un objeto B para funcionar.
- **Composición:** es el caso en el que un objeto A “es dueño de” un objeto B, y B no tiene razón de existir sin A. Esta relación es mucho más fuerte que la de agregación, ya que en este caso la parte no tiene sentido sin el todo.

Ordinalidad: ejemplo de agregación

Ejemplo:

- Un Auto tiene un Motor que le permite funcionar.
- Una computadora tiene un Mouse.

Ejemplo de código agregación:

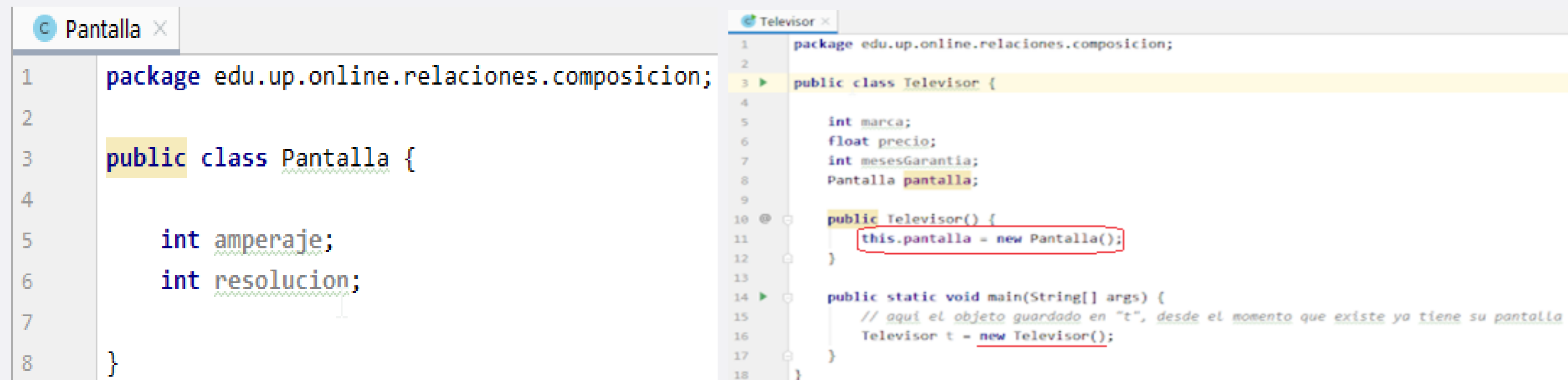
```
Auto x
1 package edu.up.online.relaciones.unoAuno;
2
3 /**
4  * Relacion 1-a-1 de Auto con Motor
5  */
6 public class Auto {
7     |
8     Motor motor;
9
10    int velocidadMaxima;
11    String marca;
12 }
```

Ordinalidad: ejemplo de composición

Ejemplo:

- Un engranaje tiene dientes, los dientes por sí solos no tienen sentido, por ende, un engranaje sin dientes no tiene razón de existir.

Ejemplo de código de composición: suponer una cadena de venta de electrodomésticos. Para ingresar el stock de televisores puedo definir esta clase televisor. De la forma en que está definida, desde el momento que existe un televisor, tiene su pantalla. Nunca puede haber un televisor SIN su pantalla (no hay ninguna tienda que venda televisores sin pantalla).



```
1 package edu.up.online.relaciones.composicion;
2
3 public class Pantalla {
4
5     int amperaje;
6     int resolucion;
7
8 }

1 package edu.up.online.relaciones.composicion;
2
3 public class Televisor {
4
5     int marca;
6     float precio;
7     int mesesGarantia;
8     Pantalla pantalla;
9
10    public Televisor() {
11        this.pantalla = new Pantalla();
12    }
13
14    public static void main(String[] args) {
15        // aquí el objeto guardado en "t", desde el momento que existe ya tiene su pantalla
16        Televisor t = new Televisor();
17    }
18 }
```



© **Universidad de Palermo**

Prohibida la reproducción total o parcial de imágenes y textos.